BSc (Hons) in **Computer Science**

**Faculty of Computing**

# SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

## Practical 02

> **Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

## Part 1: Practical Implementation — Coding the Architectures

### Step 1.1: Setting the Trap (Partial Observability)

> To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates ( `agent_pos` ). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

### Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

### Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

## Part 2: Theoretical Evaluation & Lecture Mapping

> Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

**1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?**

A Table-Driven Agent selects actions by looking up complete percept sequences in an explicitly pre-computed table. Mathematically, if an agent receives |P| distinct percepts per step and operates over a total lifetime of T time steps, the lookup table size required to map every potential sequence grows exponentially according to |P|^T. In complex environments like Chess—which features roughly 10^40 legal board states and an estimated 10^120 possible game sequences—storing such a lookup table is physically impossible because no physical storage medium in the universe possesses sufficient capacity, and no programmer could construct it. As the agent's operational lifetime T increases, the exponential growth of |P|^T causes a combinatorial explosion that rapidly exceeds all available memory and computational limits, rendering table-driven designs unviable for long-term real-world applications.

**2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.**

In the SimpleReflexAgent code, the Condition-Action Rules (IF-THEN rules) are directly implemented inside the decision control block of sense_and_act():

```
if percept.get('food_here'):
    return 'suck'
elif percept.get('wall_ahead'):
    return 'turn_left'
else:
    return 'move_forward'
```

These lines evaluate immediate sensor flags (food_here, wall_ahead) as preconditions and immediately output a corresponding actuator signal (suck, turn_left, move_forward). This directly reflects the core definition of a Simple Reflex Agent: mapping current percepts straight to actions without evaluating past history or computing future world states.

**3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?**

The SimpleReflexAgent falls into an infinite loop due to the combination of partial observability and zero percept memory. Because sensor inputs are restricted to local environment indicators, the agent is blind to its absolute global coordinates on the map. Concurrently, because the class lacks internal history variables, two identical local sensor readings always yield the exact same deterministic action. When the agent enters a corner or dead-end (State S_1), its rule outputs an action (Action A_1) that moves it into State S_2. At S_2, the local percept triggers Action A_2, which bounces the agent straight back into State S_1. Without historical memory to realize it was just at S_1, the agent endlessly repeats the cycle S_1 -> A_1 -> S_2 -> A_2 -> S_1, trapping it in an inescapable loop.

**4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).**

The ModelBasedAgent integrates a Transition Model and a Sensor Model within its update_state() method and spatial calculations to maintain an accurate internal memory of the world. The Transition Model—which represents how the world evolves—is implemented by computing expected coordinate shifts based on current heading vectors: dx, dy = self.directions[self.facing_index] and front_pos = (self.current_pos[0] + dx, self.current_pos[1] + dy). This projects how an agent's orientation translates into physical movement within the grid. The Sensor Model—which tracks how actions affect the world and sensor percepts—is implemented by evaluating actual movement outcomes before updating state memory: if self.last_action == 'move_forward' and not percept.get('hit_wall', False): self.current_pos = .... This ensures internal position tracking (current_pos) and memory sets (visited_cells) accurately reflect physical reality, allowing condition-action logic to query memory and safely navigate around previously visited locations.

Finalize and Lock Lab 02 Documentation

**Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.**

FACULTY OF COMPUTING